evangrin@ucsd.edu UC San Diego P.O. Box 1212 La Jolla California USA 92093

ECE 273 Project: Blind Deconvolution

Eric Van Grinsven

Introduction: Blind deconvolution is the problem of recovering two signals given the output of their convolution. The signals can not be recovered unless certain restrictions are imposed on them. In this project, you will study and implement convex algorithms for solving the blind deconvolution problem under suitable conditions, and evaluate their performance guarantees.

1 Section 1

Study the paper by Ahmed, Recht and Romberg to understand the formulation of the blind deconvolution problem. Why is the solution to the problem non-unique in general? Can you explain what kind of ambiguities exist? Under what condi- tions on the signals (sparsity, subspace structure etc), the problem can permit unique solutions? Explain why.

The blind deconvolution ploblem can be stated as follows: Given as input a signal y, return x and w such that y is the convolution of x and w. In general, the solution y to the blind deconvolution is non-unique. That is, there exists x, x', w, w' such that $y = w * x$ and $y = w' * x'$.

The circular convolution is defined in the paper as

$$y[l] = \sum_{i=1}^{L} w[i]x[l - i + 1]$$

where we take the modulo of $l - i + 1$.

Without restrictions on w and x, we can see from this formula of the circular convolution that the solution is in general not unique. For any entry $y[l]$, it defined as the dot product of shifted versions of the signals x and w, which can be rearranged in such a way that we can get multiple instances of x and w that satisfy $y = x * w$. We will show this in the next paragraphs.

For the remainder of this paper, we will use the following definitions:

$$w = Bh, x = Cm$$

where $B \in R^{LxK}$ and $C \in R^{LxN}$. B and C are a basis for the subspaces that w and x live in, respectively. If we assume that we are given B, C, the paper derives that while the deconvolution problem isn't linear in x, w, it can be rewritten as a problem linear in the outer product of h and m, which we write as X_0. It is clear that if we can recover h, m, then we can recover x, w by multiplying Bh, Cm respectively. We will also define $\hat{z}$ as the frequency domain representation of an arbitrary vector z. Clearly if we have $\hat{z}$ we can transform the vector back into z.

The problem can restated as $\hat{y} = A(X_0)$ for some linear operator A, which we will derive in section 2.

The outer product can be written as

$$u_1 v ... u_n v$$

, where u_i is the ith element of u, which is clearly just scaled versions of v for the outer product of v,u. Thus, X_0 is a rank-1 matrix, and so all vectors in X_0 are linear combinations of each other. Following this, consider $u' = au, v' = (1/a)v$ for any non-zero real number a. Then $u \otimes v = u' \otimes v'$

$\hat{y} = A(X_0) = A(m \otimes h) = A(m' \otimes h')$, where h',m' are defined as in the previous paragraph, which shows non-uniqueness of the problem in general. Thus, when a solution is returned, it is ambiguous what the intended constants on the input vectors x and w were supposed to be, and so we can only recover them up to a constant factor.

According to the paper, the following conditions on x, w can permit unique solutions:

1. $w, x \in R^L$ (given $y \in R^L$)
2. One of the signals, w, is in a fixed subspace of R^K and is spread out in the frequency domain.
3. x lives in a generic (but fixed) subspace in R^N chosen randomly.

By "spread out in the frequency domain", the author is discussing how jointly concentrated the columns b_l of B are in the frequency domain. The measurement

$$u_h^2 = Lmax_{1 \leq l \leq L} |\hat{w}(l)|^2 = Lmax_{1 \leq l \leq L} | < h, \hat{b}_l > |^2$$

is used to represent this.

There are other conditions that can lead to uniqueness, but these are the ones focused on in the paper, and thus I will not go into more detail.

2 Section 2

Why is the problem non-convex? Explain the main idea in this paper under which the measurements can be expressed as a linear function of a suitable transformed variable. How is this transformed variable related to the constituent signals, and given this transformed variable, how can one obtain the constituent signals? Does the variable transformation make the problem convex? What additional relaxations are necessary to cast it as a convex problem?

The problem is non-convex because $\hat{y}(l) = <\hat{c}_l, u><v, \hat{b}_l>$ itself is non-convex. We have learned in class that the inverse of a convex function is convex, and thus $\hat{y}^{-1}$ is not convex, since it is was then $\hat{y}$ would be convex, a contradiction.

Now, we will derive the main formulas from section 2, along with more explanation and intuition into the provided derivation.

As defined before, we have $x = Cm = m(1)C_1 + ... + m(N)C_N$, where C_i is defined to be the ith column of C, a basis for x. Thus we can rewrite

$$y = w * x \ (1)$$

$$y = w * (m(1)C_1 + ... + m(N)C_N) \ (2)$$

$$y = w * (m(1)C_1) + ... + w * (m(N)C_N)) \ (3)$$

$$y = m(1)w * C_1 + ... + m(N)w * C_N \ (4)$$

y[l] cannot be expressed as a linear function of x and w. For this to happen, we would need to write $y[l] = Px + Qy + c = PCm + QBh + c$, but instead we have equation (2), which is clearly not linear in m,h, which implies it is not linear in x,w. Thus y itself is not a linear function w.r.t. x and w.

I did research into circular matrices and confirmed for myself that $x * y$ can be rewritten as $circ(x)y$, where $circ(x)$ is the circular matrix of x (where it is assumed we are taking the circular convolution). For brevity, I will not include a derivation, as it is assumed the readers of this paper are familiar with this operation.

$$y = m(1)circ(C_1)w + ... + m(N)circ(C_N)w \ (5)$$

$$y = m(1)circ(C_1)Bh + ... + m(N)circ(C_N)Bh \ (6)$$

$$y = circ(C_1)Bm(1)h + ... + circ(C_N)Bm(N)h \ (7)$$

$$y = [circ(C_1)B + ... + circ(C_N)][m(1)h...m(N)h]^T \ (8)$$

Converting it to the Fourier domain gives us

$$\hat{y} = Fy = [\Delta_1\hat{B} + ... + \Delta_N\hat{B}][m(1)h...m(N)h]^T \ (9)$$

(9) comes from the following facts - $\hat{y} = Fy$ where F is defined to be the matrix that transforms the vector y into it's frequency representation. A similar definition is applied to the matrix B. By defining Δ_i to be the ith column of $\sqrt{L}\hat{C}$, diagonalized we also have that $circ(C_i) = F^*\Delta_i F$ through some algebraic manipulation.

(9) also clearly shows $\hat{y}$ is not linear wrt m and h. Since it is not linear wrt m and h, it cannot be linear wrt x and w, since $x = Cm$ and $w = Bh$.

However, the matrix $[m(1)h...m(N)h]$ is simply the outer product of h and m. As written in Section 1, we have $X_0 = m \otimes h$. Using (9), we can write

$$\hat{y} = MX_0 \ , \ M = [\Delta_1\hat{B} + ... + \Delta_N\hat{B}] \ (10)$$

Thus, $\hat{y}$ is linear in $m \otimes h$. We can write it as

$$\hat{y} = A(X_0) \ (11)$$

where A is a linear operator.

Given this transformed variable of x, w, we can recover x, w up to a constant. I described this in Section 1, but will include my argument here for completion.

The outer product can be written as

$$u_1 v ... u_n v$$

, where u_i is the ith element of u, which is clearly just scaled versions of v for the outer product of v,u. Thus, X_0 is a rank-1 matrix, and so all vectors in X_0 are linear combinations of each other. Following this, consider $u' = au, v' = (1/a)v$ for any non-zero real number a. Then $u \otimes v = u' \otimes v'$.

The variable transformation does not make the problem convex since the set of rank-1 matrices in non-convex. However, we will make the following relaxations to cast it as a convex problem.

They first define the optimization problem:

$$min_{u,v}||u||_2^2 + ||v||_2^2 \ subject \ to \ \hat{y}(l) = <\hat{c}_l, u><v, \hat{b}_l>, l = 1, .., L \ (12)$$

(12) is non-convex because of the quadratic equality constraints of the feasible set. EXPLAIN WHY. The dual of (12) is shown below.

$$max_\lambda Re < \hat{y}, \lambda >$$

subject to

$$IZZ^*I \geq 0 \ (13)$$

where $Z = \sum_{l=1}^{L} \lambda(l)\hat{b}_l\hat{c}_l^*$

If we take the dual again, we are left with

$$min||X||_* \ subject \ to \ \hat{y} = A(X) \ (14)$$

The authors state that this is "a convex program which is in some sense as close to (11) as possible." (14) is the "dual-dual" of the non-convex least-squares problem, as seen in (11). This is convex as we have shown that the dual is convex. Convexity is also clear from the formulas since we have shown $\hat{y} = A(X_0)$ is linear in terms of X_0, and the nuclear norm, as with all norms, is convex. Thus, the dual dual relaxations are necessary to cast it as a convex problem.

3 Section 3

What are the theoretical guarantees of the convex algorithm proposed in the paper? Are they deterministic, or probabilistic? What should be the relation of the number of measurements, sparsity and the dimension of the subspace, so that exact (or stable, in presence of noise) recovery is possible? Clearly state the mathematical guarantees and spell out the assumptions.

The theoretical guarantees of the convex algorithm from the paper are probabilistic.

The following paragraphs will be used to specify and define terms that are used for our assumptions in the theoretical guarantees.

Firstly, w is time limited to $K \leq Q \leq L$, which implies that the last L-Q rows of B are 0. In addition, we assume the columns of B are orthonormal.

Next, we define what it means for B to be "diffuse in the Fourier domain." "The (in)coherence pattern u_{max} quantifies the degree to which the columns of B are jointly concentrated in the Fourier domain."

$$u_{max} = \frac{L}{K} max ||\hat{b}_l||_2^2 \ (14)$$

. Let's look at the intuition behind this formula. The total energy from the rows in B is given by

$$\sum_{l=1}^{L} = ||\hat{b}_l||_2^2 = K \ (15)$$

, where equality comes from our assumption of orthonormality. We also know that $||\hat{b}_l||_2^2 \leq 1$ from orthonormality. These both imply

$$1 \leq u_{max}^2 \leq \frac{L}{K} \ (15)$$

.

We also define

$$u_{min}^2 = \frac{L}{K} min_{1 \leq l \leq L} ||\hat{b}_l||_2^2 \ (16)$$

and with similar arguments $0 \leq u_{min}^2 \leq 1$ and $u_{min}^2 \leq u_{max}^2$.

The value u_h will represent how diffuse the signal w is in the frequency domain.

$$u_h^2 = Lmax|w(l)|^2 = Lmax|\langle h, \hat{b}_l \rangle|^2 \ (17)$$

I will not derive the next equation, but it will be used in our representation of diffusion of the signal in the freq dom.

$$1 \leq u_h^2 \leq u_{max}^2 K \ (18)$$

As described in the paper "if the signal is more or less 'flat' in the frequency domain, then u_h^2 will be a small constant."

Our last assumption is that we choose a subspace of dimension N at random for C from an isotropic distribution.

$$c[l, n] \ Normal(0, L^{-1} \ (19)$$

The analysis assumes that the rows of $\hat{C}$ are independent.

We have now stated all of the assumptions needed for the main theoretical results. The two theoretical results below are probabilistic.

(Theorem 1):

In addition to the paragraphs above, assume $Q \geq C_\alpha M log(L) log(M)$ where Q is defined above and $M = max(u_{max}^2 K, u_h^2 N$. In addition, assume L is a multiple of M and $\frac{L}{Q} \geq log(C'_\alpha \sqrt{NlogL}/log2$.

Then $\exists\, C''_\alpha \in O(\alpha)$ such that if

$$max(u^2_{max}K, u^2_h N) \leq \frac{L}{C''_\alpha log^3 L}$$

then $X_0 = hm^*$ is the unique solution to (11) with probability $1 - O(L^{-\alpha+1})$.

By our earlier discussion, we have already shown that given $X_0 = hm^*$, we can recover h and m up to a scalar multiple. Clearly, this implies that we can recover x,w up to a scalar multiple, since $x = Cm$, $w = Bh$.

Our next theorem now considers probabilistic guarantees of the deconvolution problem in the presence of noise. More formally, we observe

$$\hat{y} = A(X_0) + z$$

, where $z \in R^L$ is unknown noise and $||z||_2 \leq \delta$. We estimate X_0 by solving

$$min\ ||X||_*$$

$$subject\ to\ ||\hat{y} - A(X)||_2 \leq \delta\ (20)$$

(Theorem 2):

Let $\lambda_{min}, \lambda_{max}$ be the largest/smallest non-zero eigenvalues of AA^* respectively. Then with probability $1 - O(L^{-\alpha+1})$, the solution $\tilde{X}$ to (20) satisfies (for fixed C)

$$||\tilde{X} - X_0||_F \leq C\delta \frac{\lambda_{max}}{\lambda_{min}} \sqrt{min(K, N)}\ (21)$$

In addition, if $NK \geq \frac{C_\alpha}{u^2_{min}} Llog^2 L$, then whp theorem 2 still holds when we replace $\frac{\lambda_{max}}{\lambda_{min}}$ with $\frac{u_{max}}{u_{min}}$.

4 Section 4

Implement the convex algorithm proposed in the paper. You should be able to recre- ate the figures. Test the algorithm by generating the phase transition plots which plots the probability of successful recovery as a function of number of measure- ments, and the sparsity and/or dimension of the subspace. Do you see a sharp region around which the probability changes from close to zero, to close to 1? Com- pare this with the theoretical guarantees from the paper.

For figures 4 and 5, we define a success as

$$\frac{||\hat{X} - wx^*||_F}{||wx^*||_F} \leq .02\ (A)$$

where the left-hand side is defined to be the relative error of our evaluation.

Figure 3:

For the following experiment, phase diagrams were used "to demonstrate the empirical probability of success over a range of dimensions N and K for a fixed L" (page 12). x is a random vector in the subspace spanned by the columns of an LxN matrix whose entries are independent and identically distributed Gaussian random variables. In (a), w is a generic sparse vector with support and non-zero entries chosen randomly. In (b), w is a generic short vector whose first K terms are non-zero and chosen at random. Due to computational limitations, I had to slightly alter the simulations. I chose L=64, and ranged K,N between 1 and 32. Whereas the paper performed each experiment 100 times and averaged the results, I did each 5 times.

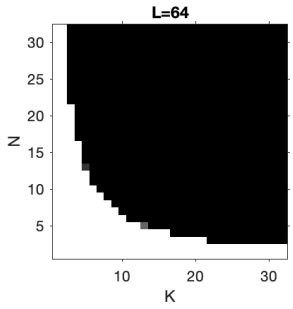


Figure 1: Figure 3a from paper. x is a random vector in the subspace spanned by the columns of an L × N matrix whose entries are independent and identically distributed Gaussian random variables. w is a generic sparse vector, with support and nonzero entries chosen randomly.

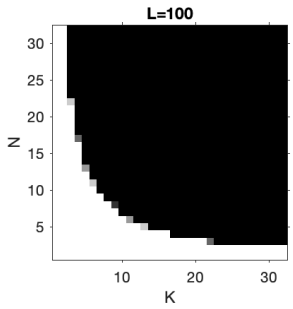


Figure 2: Figure 3b from paper. x is a random vector in the subspace spanned by the columns of an L × N matrix whose entries are independent and identically distributed Gaussian random variables. w is a generic short vector whose first K terms are nonzero and chosen randomly.

Similar to the paper, I saw a sharp region around where the probability changes from close to 0 to close to 1. However, the ratio where I saw these changes were higher than the paper. Whereas they saw it around L/(N+K) = 2.7, I saw mine in the range from 3 to 4 in both my 3a and 3b experiments.

Figure 4:

In figure 4, the experiments are similar, except x/w are randomly chosen sparse vectors. In (a), B and C are generated by randomly selecting columns of the identity. In (b), C is chosen the same, but B is chosen to have the first K rows be the identity, and the remaining L-K rows be empty. In my opinion, the paper wasn't entirely clear on how they performed (b) as they state "we use the same model for C and x, but use a "short" w (first K terms are non-zero)." However, they also said that they "w is a generic short vector whose first K terms are nonzero and chosen randomly.". I see this as ambiguous whether they choose values from a Gaussian, or just choose a subset of the identity LxL columns. For both (a) and (b), I chose h and m from as a gaussian random vector with mean and variance 0 and 1 respectively. Due to computational limitations, I had to slightly alter the simulations. I chose L=100, and ranged K,N between 1 and 32. Whereas the paper performed each experiment 100 times and averaged the results, I did each 5 times.

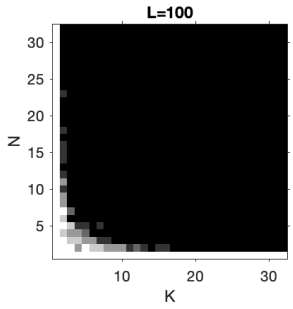


Figure 3: Figure 4a from paper. x is a random sparse vector whose support and N non-zero values on that support are chosen at random. w is a generic sparse vector, with support and K nonzero entries chosen randomly.

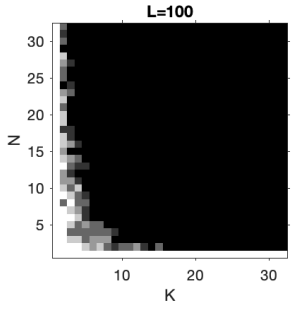


Figure 4: Figure 4b from paper. x is a random sparse vector whose support and N non-zero values on that support are chosen at random. w is a generic short vector whose first K terms are nonzero and chosen randomly.

Again, I saw a sharp region around where the probability changes from close to 0 to close to 1. This time the ratio was much higher than the expected 2.7 like the paper. The sharp change is about 7.1 a the bottom of the arc.

Figure 5:

We now set $y = w * x + z$, where $z \in R^l$ is a gaussian random vector with mean 0 and variance σ^2. As done in the paper, we solve (19) with $\delta = \sigma\sqrt{(L + \sqrt{4L})}$. We chose B to be a random selection of columns from the LxL identity, and C to be iid Gaussian matrix. Finally, we chose h and m to be gaussian vectors with mean 0 and variance 1 for each experiment.

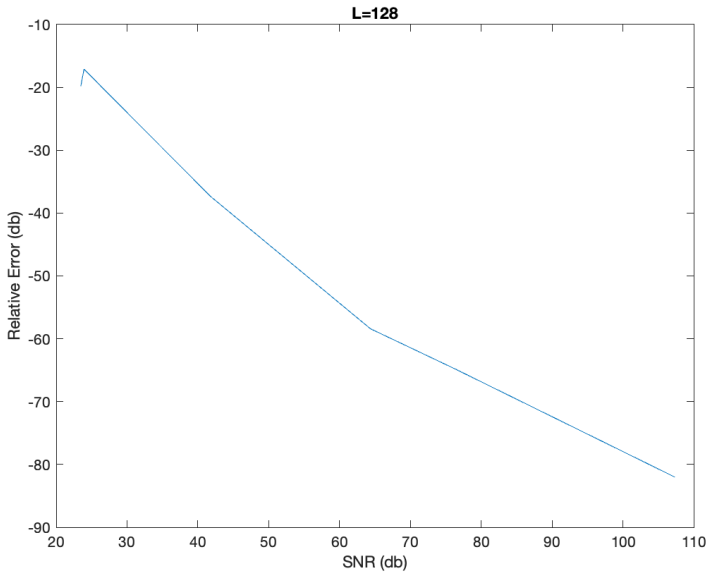


Figure 5: Figure 5a from paper. Relative error vs. SNR on a log-log scale.

(a) compares the relative error versus the changes to noise level. We use the definition of

$$SNR = 10log_{10}(||wx^*||_F^2/||z||_2^2)(23)$$

. In addition, we will make this a log-log plot by taking the y axis to be $10log_10(rel\ error)$, where relative error is defined in the lhs of (A). I only perform the algorithm on each input exactly once, and so we will not get the averaging that we've been doing thus far.

The paper found a linear relationship between the recovery error and the signal-to-noise ratio. The larger the noise, the larger the error. In my experiment, I had similar findings. The relative error (in db) was about -20 with an SNR of about 30, which lines up with the paper. From there, however, the error (while still a linear relationship) did not match the slope in the paper. Specifically, the slope in the paper was about -1.35 (I estimated by choosing 2 points, since we were not given exact data). In mine, we had about -.65. One reason I can think of is that I did not average my results as they did with 100 attempts each time.

(b) compared the recovery error and the oversampling ratio, defined by $L/N + K$. They found that as the ratio increased (ie L became much larger than N+K) the recovery error decreased. Specifically, they found significant dropoff once the ratio increased from 1 to between 2 and 3. This was expected, since the paper mentioned that experimentally they found that when $L = 2.7(N + K)$, they found the recovery error to drastically change.

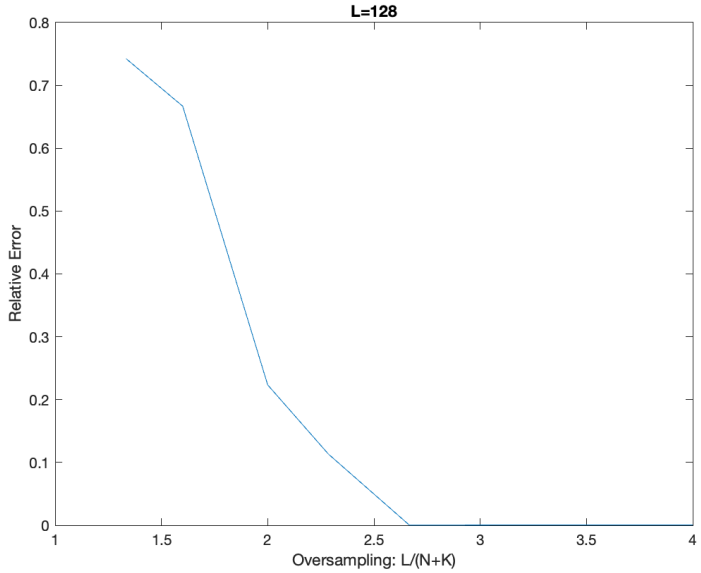


Figure 6: 5b from paper. Oversampling rate vs. relative error for a fixed SNR of 20dB

To perform this experiment, I fixed L=128, and experimented with oversampling ratios ranging from 1-4. (Doing more than 4 made the graph hard to read, as was done in the paper). I fixed $\sigma = 0.1$ for the noise ratio, which experimentally gave close results to the SNR of 20dB that the paper used for this experiment. Finally, I ran the chosen pairs of (K,N) 5 times and took the average for more accurate results (again, due to computational constraints, I could not do the full 100 as done in the paper).

I had similar experimental results to that of the paper. I saw significant dropoff from an oversampling ratio between 1.5 and 3, which is is similar to the paper. I saw the relative error drop to near 0 when we hit about 2.73, which was expected.

5 Section 5

Compare performance of the blind deconvolution algorithm against non-blind de- convolution (i.e. where you are given one of the constituent signals). Do a literature survey on alternating minimization techniques for blind deconvolution (which are non-convex), and implement one algorithm of your choice. Compare the perfor- mance of convex and non-convex blind deconvolution.

Let's discuss what happens when one of the constituent signals is known. WLOG, assume x is our known signal, and we want to find w. We will show that equation (9) is convex in h. Since we are given x, we can rewrite (9) as

$$\hat{y} = Fy = m(1)\Delta_1 \hat{B}h + ... + m(N)\Delta_N \hat{B}h \ (22)$$

, which is a linear combination of h, and so it is convex. Clearly, if we find h, then we can perform $w = Bh$ to recover w. In addition, blind deconvolution led to our solution only being accurate up to a constant. However, with this unblind convolution we no longer have this issue, as knowing one vector in an outer product allows us to recover the other element (whereas before this would only be accurate up to a scalar constant).

Due to the above, we would expect a simple convex optimization formulation to solve for h. I wrote the following algorithm:

$$min \ circ(x)w - y \ (22)$$

Since

$$circ(x)w = x * h$$

, we are simply minimizing the convolution of x with our estimation of w with the actual value of the convolution.

Our experimental results for non-blind deconvolution can be found in Figure 6. We performed the same basic setup as with 3 and 4, and did L=64 and ranked N,K between 1 and 64. For the relative error, we are simply calculating the l2 distance of our predicted value for w and the actual value. Using the same success criteria as before in (A), we were able to solve for x successfully in each experiment. In addition, we have plotted the relative error compared to the relative error of blind deconvolution in figure 7. For both experiments, we chose L = 64 and fixed the size of w to 8, and ranged x from 1 to 32. We can clearly see that the error of the non-blind deconvolution was much lower for all of the values experimented with. Particularly, the error doesn't rise with the ratio of L/(N+K) increases, and I attribute this to the convexity of the problem itself.

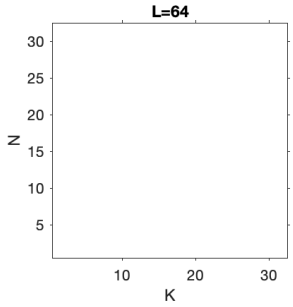


Figure 7: Same general experiment as my figures 1-4, except here we only recover x. Details explained in writeup above.

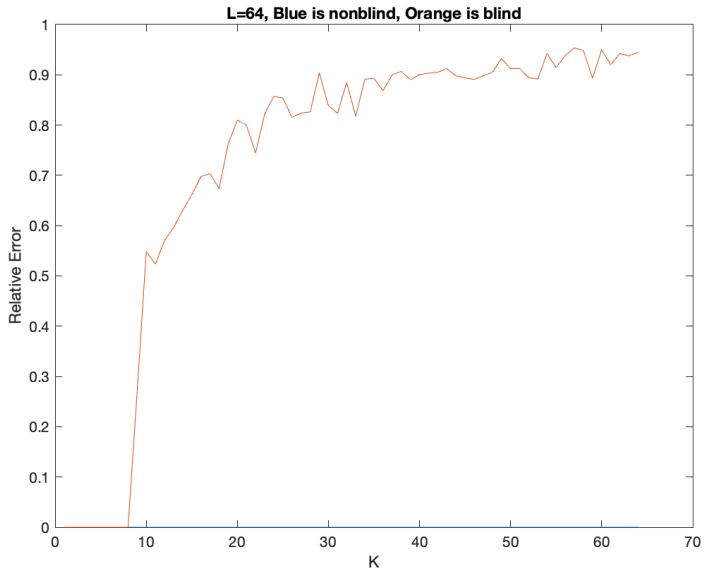


Figure 8: Orange is blind deconvolution recovering x and w, Blue is non-blind recovering x. Note that we are comparing relative error for both, however the error for nonblind is just the relative error for one vector, x, whereas the relative error for blind is the outer product of x and w.

Alternating Minimization:

To be fully transparent, I did not get my Alternating minimization implementation working correctly. I attempted to follow the algorithm in "Total Variation Blind Deconvolution: The Devil is in the Details". I searched through many papers on the topic, and found many of them a bit too complex to grasp in my limited time remaining. This is especially true for projected alternating minization which I read a bit about. I decided to implement the first algorithm in the paper listed above which seems to just be pure alternating minization. I did not implement anything special for initialization, and simply choose to assume k^0 to be a random gaussian with 0 mean and variance 1.

I would still like to mention that I think the idea of alternating minization is quite intuitive at the surface level (unless I'm misunderstanding it), where at every step you are just updating one of the signals based on your previous estimation of the other signal, and continue this process until your convergence criteria.

3.2. Exact Alternating Minimization

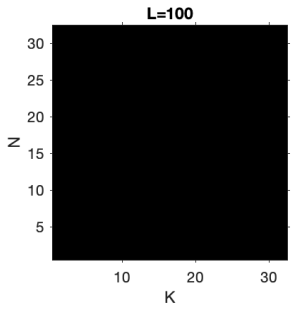
A solution to problem (6) can be readily found by an iterative algorithm that alternates between the estimation of the sharp image given the kernel and the estimation of the kernel given the sharp image. This approach, called *alternating minimization* (AM), requires the solution of an unconstrained convex problem in u

$$u^{t+1} \leftarrow \min_u ||k^t * u - f||_2^2 + \lambda J(u) \tag{10}$$

and a constrained convex problem in k

$$\begin{aligned} k^{t+1} \leftarrow \quad & \min_k \quad && ||k * u^t - f||_2^2 \\ & \text{subject to} \quad && k \geqslant 0, \quad \|k\|_1 = 1. \end{aligned} \tag{11}$$

Unfortunately, as we show next, this algorithm suffers from the limitations brought out by Theorem 3.1, and without a careful initialization could get stuck on the no-blur solution.



6 Section 6

Design numerical experiments to evaluate the performance of the algorithm when one (or more) conditions (such as sparsity, low rank) are violated. Can you comment on the robustness of the

algorithm against such violations?

Our experiment compares low rank We take C to be a dense Gaussian random matrix, and we take B to be the first K columns of the LxL identity matrix. In this experiment, we want to compare the relative error versus the size of K. We fixed L=64 and N=8. We found the error drop to near 0 when K=8, which gives the $L/(N+K)$ ratio of 4. This seems to correspond with my phase diagram experiments and discussion above.

When we violate the low rank assumption, however, we see the error shoot up after this rank, showing experimentally that when low rank is violated, we lose error guarantees.

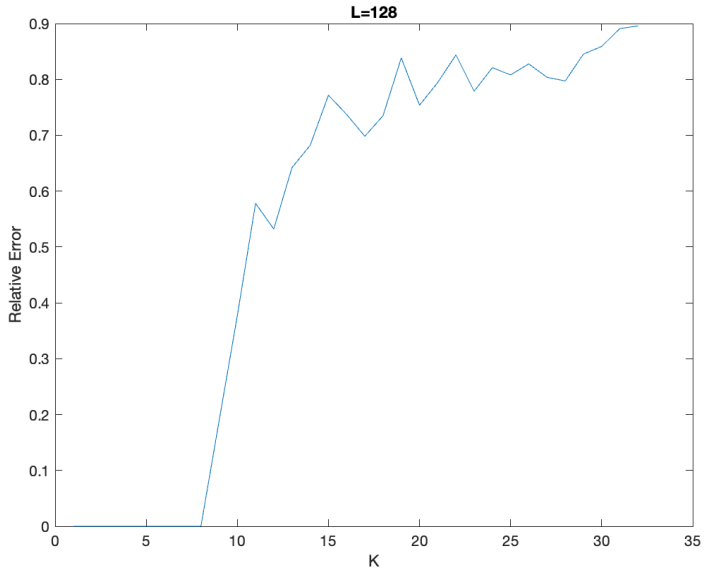


Figure 9: Low Rank Experiment. Typo: L = 64

7 Section 7

Summarize your overall conclusions about what you learned from this project.

To be honest, I was surprised with the accuracy with most of my findings, since all of my figures looked similar to the ones in the paper. I didn't have prior experience reimplementing work done in papers, and so I'm happy with my findings.

For figures 3a-4b in the paper, I saw similar results, however my ratio's for success tended to be higher. In addition, I saw a significant gap in the performance between my experiments between 3 and 4 in the original paper, whereas they didn't.

For part 5a I saw the linear relationship as seen in the paper, although I did have a different slope, which I attribute mostly to not performing multiple trials for each input. For part 5b I got very similar results to the paper, and saw the drop in error very close to that in the paper, with close to 0 error when we hit an oversampling ratio of about 2.75.

We also looked at non-blind deconvolution and showed that this problem was convex, and wrote a simple convex optimization algorithm to solve for the unknown signal. We also saw experimentally that the error of our resulting signal was in general much lower than the error for blind deconvolution.

We also looked at another paper on alternating minimization techniques for solving the blind deconvolution problem, and unfortunately my results are not telling in the slightest.

Finally, I evaluated the performance of the algorithm when one (or more) conditions were violated, such as low rank or sparsity. I found that the convex algorithm is not extremely robust when conditions are violated.

Now, here are some of my overall opinions on the project. I personally chose this project because I don't have any experience with signal processing and so I wanted to get an introduction into the field. I have past experience with Neural Networks, but I do not plan on pursuing a career in ML, and so my interests led me to the blind deconvolution problem. First and foremost, I learned new approaches in reading papers.

My background has me reading a lot of papers in the field of Computer Systems and Networking, which are often less theoretical and mathematical in nature than a paper like this. Thus, I initially felt bogged down by all of the math involved. In addition, I have a less than stellar background in Linear Algebra and with convolutions and other signal processing necessities. Thus, it was a big learning curve to jump into reading the proofs and getting my mathematical background up to date. I spent a lot of time writing out the formulas in the first section and varifying their correctness for myself. However, at a certain point I realized that there was too much information for me to fully grasp in the provided timeline, and so I had to learn what was important for my understanding, and what I could accept without needing to fully understand. It is clear to me that this is a necessity, at least for the earlier stages of a career in academia, since it is not possible (for me at least) to have knowledge in so many domains of mathematics and engineering.

Once I accepted that there were some portions out of my scope, I was able to focus on what I found important, which were the initial formulations, the results and analysis, and approaches to solving the problems within the domain of convex optimization.

Unfortunately, I ran out of time to do some of the experiments like AM the justice that it deserved, and so my results there were unfortunate and I take full responsibility for that.

Finally, here is the github link to all of my code. https://github.com/EricVanGrinsven/ECE273Code

8 Bonus

Extend the algorithm to perform blind deconvolution in 2D. One example is image deblurring. Test the algorithm on an image of your choice (by blurring it with a suitable kernel)

I spent a bit of time attempting this, but did not have the time to delve into techniques the paper talked about like estimating the basis through wavelets. I just attempted to model it similarly to the previous questions and stacking my 28x28 image of a boot into a column, but my result was a black 28x28 image, so I will not post it here. I will attach the code to show that I at least attempted it, but not much progress was made. The file is bonus.m.